

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	D. Gnana Deepthi	Reg.No.:	192411123
Department:	CSE	Date:	

ANNEXURE A

Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I D. Gnana Deepthi (192411123) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D. Gnana Deepthi

ANSWERS

1. DMA-Based Data Transfer Algorithm

Pseudocode

ALGORITHM DMA_Data_Transfer

INPUT:

I/O_Device

Memory_Address

Data_Size

OUTPUT:

Data transferred from I/O device to memory

BEGIN

// Step 1: CPU initializes DMA controller

CPU:

Set DMA.Source $\leftarrow$ I/O_Device

Set DMA.Destination $\leftarrow$ Memory_Address

Set DMA.Transfer_Count $\leftarrow$ Data_Size

Set DMA.Direction $\leftarrow$ I/O_TO_MEMORY

Set DMA.Mode $\leftarrow$ BLOCK_TRANSFER

// Step 2: Enable DMA interrupt

DMA.Interrupt_Enable $\leftarrow$ TRUE

// Step 3: Start DMA transfer

DMA.Enable $\leftarrow$ TRUE

// Step 4: CPU is free to perform other tasks

CPU:

Perform_Other_Tasks()

// Step 5: DMA controller requests access to memory

DMA:

Request_Bus()

IF Bus_Granted = TRUE THEN

WHILE Transfer_Count > 0 DO

Read data from I/O_Device

Write data to Memory_Address

Memory_Address $\leftarrow$ Memory_Address + 1

Transfer_Count $\leftarrow$ Transfer_Count - 1

END WHILE

// Step 6: DMA transfer completed

DMA.Enable $\leftarrow$ FALSE

DMA.Release_Bus()

// Step 7: Generate interrupt

IF DMA.Interrupt_Enable = TRUE THEN

 Generate_Interrupt()

END IF

END IF

// Step 8: CPU handles DMA completion interrupt

CPU:

 Save_Current_State()

 Check_DMA_Status()

IF DMA_Status = TRANSFER_COMPLETE THEN

 Acknowledge_Interrupt()

 Set_Transfer_Status $\leftarrow$ SUCCESS

ELSE

 Handle_DMA_Error()

 Set_Transfer_Status $\leftarrow$ FAILURE

END IF

Restore_Previous_State()

Resume_Other_Tasks()

END

2. Interrupt Handling System Using Vectored Interrupts

Pseudocode

ALGORITHM Vectored_Interrupt_Handler

INPUT:

Interrupts from multiple devices

OUTPUT:

Appropriate Interrupt Service Routine executed

BEGIN

// Step 1: Initialize interrupt vector table

Initialize Interrupt_Vector_Table

Interrupt_Vector_Table[0] ← Timer_ISR

Interrupt_Vector_Table[1] ← Keyboard_ISR

Interrupt_Vector_Table[2] ← Disk_ISR

Interrupt_Vector_Table[3] ← Network_ISR

Interrupt_Vector_Table[4] ← I/O_Device_ISR

// Step 2: Set interrupt priorities

Set Priority(Timer) ← HIGH

Set Priority(Network) ← HIGH

Set Priority(Disk) ← MEDIUM

Set Priority(Keyboard) ← LOW

Set Priority(I/O_Device) $\leftarrow$ LOW

Enable_Global_Interrupts()

// Step 3: Continuously monitor for interrupts

WHILE System_Running = TRUE DO

IF Interrupt_Request_Detected() = TRUE THEN

 // Step 4: Identify pending interrupts

 Pending_Interrupts $\leftarrow$ Get_Pending_Interrupts()

 // Step 5: Select highest-priority interrupt

 Selected_Interrupt $\leftarrow$ Select_Highest_Priority(
 Pending_Interrupts)

 // Step 6: Obtain interrupt vector number

 Vector_Number $\leftarrow$ Get_Vector_Number(
 Selected_Interrupt)

 // Step 7: Save current CPU state

 Save_CPU_Registers()

 Save_Program_Counter()

 Save_Status_Register()

 // Step 8: Dispatch appropriate ISR

 ISR_Address $\leftarrow$ Interrupt_Vector_Table[Vector_Number]

 Jump_To(ISR_Address)

// Step 9: Execute the selected ISR

Execute_ISR()

// Step 10: Acknowledge interrupt

Acknowledge_Interrupt(Selected_Interrupt)

// Step 11: Restore CPU state

Restore_Status_Register()

Restore_Program_Counter()

Restore_CPU_Registers()

// Step 12: Return to interrupted program

Return_From_Interrupt()

END IF

END WHILE

END

3. Dynamic Selection Between Memory-Mapped I/O and I/O-Mapped I/O

Pseudocode

ALGORITHM Select_IO_Mapping

INPUT:

Required_Latency

Required_Bandwidth

CONSTANTS:

MMIO_Max_Latency

MMIO_Min_Bandwidth

IOMapped_Max_Latency

IOMapped_Min_Bandwidth

OUTPUT:

Selected_IO_Method

BEGIN

// Step 1: Check latency requirement

IF Required_Latency <= MMIO_Max_Latency THEN

// Step 2: Check bandwidth requirement

IF Required_Bandwidth >= MMIO_Min_Bandwidth THEN

Selected_IO_Method ← MEMORY_MAPPED_IO

ELSE

Selected_IO_Method ← IO_MAPPED_IO

END IF

ELSE

// Higher latency can use I/O-mapped I/O

IF Required_Bandwidth >= IOMapped_Min_Bandwidth THEN

Selected_IO_Method ← IO_MAPPED_IO

ELSE

Selected_IO_Method $\leftarrow$ IO_MAPPED_IO

END IF

END IF

// Step 3: Configure the selected I/O method

IF Selected_IO_Method = MEMORY_MAPPED_IO THEN

Configure_MMIO()

Display("Memory-Mapped I/O Selected")

ELSE

Configure_IO_Mapped_IO()

Display("I/O-Mapped I/O Selected")

END IF

RETURN Selected_IO_Method

END

Improved Decision-Based Pseudocode

A more practical approach is to calculate a **score** for each method:

ALGORITHM Dynamic_IO_Selection

INPUT:

Required_Latency

Required_Bandwidth

BEGIN

```

MMIO_Score ← 0
IOMapped_Score ← 0

// Evaluate latency
IF Required_Latency <= LOW_LATENCY_THRESHOLD THEN
    MMIO_Score ← MMIO_Score + 2
ELSE
    IOMapped_Score ← IOMapped_Score + 1
END IF

// Evaluate bandwidth
IF Required_Bandwidth >= HIGH_BANDWIDTH_THRESHOLD THEN
    MMIO_Score ← MMIO_Score + 2
ELSE
    IOMapped_Score ← IOMapped_Score + 1
END IF

// Select method
IF MMIO_Score > IOMapped_Score THEN
    Selected_Method ← MEMORY_MAPPED_IO
    Configure_MMIO()
ELSE
    Selected_Method ← IO_MAPPED_IO
    Configure_IO_Mapped_IO()
END IF

RETURN Selected_Method

```

END

4. Bus Arbitration Algorithm for PCIe, USB, and Thunderbolt

Pseudocode

ALGORITHM Bus_Arbitration

INPUT:

Request_List from PCIe, USB, and Thunderbolt devices

OUTPUT:

Selected device granted access

BEGIN

// Step 1: Initialize interface priorities

Set Priority(PCIe) ← HIGH

Set Priority(Thunderbolt) ← HIGH

Set Priority(USB) ← MEDIUM

Initialize Round_Robin_Pointer

Initialize Device_Queues

WHILE System_Running = TRUE DO

// Step 2: Collect requests from all interfaces

PCIe_Requests ← Get_PCIe_Requests()

USB_Requests ← Get_USB_Requests()

Thunderbolt_Requests ← Get_Thunderbolt_Requests()

Add PCIe_Requests to Device_Queues
Add USB_Requests to Device_Queues
Add Thunderbolt_Requests to Device_Queues

// Step 3: Check whether requests are pending
IF Device_Queues are NOT EMPTY THEN

// Step 4: Select highest-priority request
Highest_Priority ← Find_Highest_Priority(
 Device_Queues)

// Step 5: Prevent starvation using round-robin
Candidate_List ← Requests_With_Priority(
 Highest_Priority)

Selected_Request ← Round_Robin_Select(
 Candidate_List,
 Round_Robin_Pointer)

// Step 6: Grant access
Send_GRANT(Selected_Request.Device)

// Step 7: Transfer data
Start_Transfer(Selected_Request)

WHILE Transfer_Not_Complete DO

 Monitor_Timeout()

Monitor_Error()

IF Timeout OR Error_Detected THEN

Abort_Transfer()

Handle_Error()

BREAK

END IF

END WHILE

// Step 8: Release access

Send_RELEASE(Selected_Request.Device)

Remove Selected_Request from Device_Queues

// Step 9: Update round-robin pointer

Update_Round_Robin_Pointer()

ELSE

// No device is requesting access

Enter_Idle_Mode()

END IF

END WHILE

END

5. Hybrid I/O Communication Mechanism

Pseudocode

ALGORITHM Hybrid_IO_Communication

INPUT:

Device_List

OUTPUT:

Data transferred from all devices

BEGIN

FOR each Device in Device_List DO

 // Step 1: Determine device speed

 Device_Speed ← Get_Device_Speed(Device)

 IF Device_Speed = LOW_SPEED THEN

 // Low-speed device uses polling

 Display("Polling Mode Selected")

 WHILE Device_Has_Data(Device) = FALSE DO

 Check_Device_Status(Device)

 END WHILE

 // Read data directly using CPU

 Data ← Read_From_Device(Device)

```
Store_Data_In_Memory(Data)
```

```
ELSE IF Device_Speed = HIGH_SPEED THEN
```

```
// High-speed device uses DMA
```

```
Display("DMA + Interrupt Mode Selected")
```

```
// Configure DMA controller
```

```
DMA.Source ← Device
```

```
DMA.Destination ← Memory_Buffer
```

```
DMA.Transfer_Size ← Get_Data_Size(Device)
```

```
DMA.Direction ← DEVICE_TO_MEMORY
```

```
// Enable DMA completion interrupt
```

```
DMA.Interrupt_Enable ← TRUE
```

```
// Start DMA transfer
```

```
DMA.Enable ← TRUE
```

```
// CPU performs other tasks
```

```
CPU.Perform_Other_Tasks()
```

```
END IF
```

```
END FOR
```

```
// Step 2: Handle DMA interrupts
```

```
WHILE System_Running = TRUE DO
```

```
    IF DMA_Interrupt_Detected() = TRUE THEN
```

```
        Save_CPU_State()
```

```
        Device ← Identify_DMA_Device()
```

```
        DMA_Status ← Check_DMA_Status(Device)
```

```
        IF DMA_Status = TRANSFER_COMPLETE THEN
```

```
            Acknowledge_DMA_Interrupt()
```

```
            Process_Transferred_Data(Device)
```

```
        ELSE IF DMA_Status = ERROR THEN
```

```
            Handle_DMA_Error(Device)
```

```
        END IF
```

```
        Restore_CPU_State()
```

```
        Resume_CPU_Execution()
```

```
    END IF
```

```
END WHILE
```

```
END
```

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent	Mostly clear with	Difficult to follow in parts	Poorly structured and unclear

		format, easy to follow	minor issues		
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient